THE COMPLETE HANDS-ON GUIDE

Ollama

Prompting & Conversations That Actually Work

A model is only as good as the conversation you build around it. This phase turns you from someone who sends prompts into someone who *designs* them — roles, system prompts, memory, context budgets, few-shot examples, step-by-step reasoning, and reliable JSON output.

PHASE 3 OF 6

Prompting & Conversations

Based on Ollama v0.23.x & ollama-python v0.6.x · docs.ollama.com

Edition 2026

Phase 3 — Prompting & Conversations

> In Phase 2 you learned *how* to call the model. This phase is about *what* to say to it. The same model can be vague or razor-sharp, forgetful or context-aware, chatty or perfectly structured — and the difference is almost entirely in how you construct the messages. These are the techniques that separate a toy script from a genuinely useful AI tool.

Everything here builds directly on the `chat()` function and its **messages list** from Phase 2. Recall the three roles — `system`, `user`, `assistant`. Mastering how to arrange and grow that list is the whole game.

WHAT YOU'LL LEARN IN THIS PHASE

The three roles and what each is for · writing effective system prompts · building multi-turn conversations by appending to the messages list · managing context length by trimming history · few-shot prompting · chain-of-thought reasoning · forcing structured JSON output · and packaging prompts into reusable template functions.

3.1 — The three roles: system, user, assistant

Every message in a chat carries a `role`. The role tells the model *who is speaking*, and the model treats each one differently. Getting roles right is the foundation of every other technique in this phase.

Role	Purpose	When you write it
`system`	Sets the model's identity, rules and behaviour for the whole conversation. The model treats it as authoritative, standing instructions.	Once, as the first message (optional but powerful).
`user`	The human's input — questions, requests, data to process.	Every time the person says something.
`assistant`	The model's own previous replies. Including them is how the model "remembers" earlier turns.	You append the model's answer after each turn.

```
import ollama

messages = [
    {'role': 'system',    'content': 'You are a calm, encouraging maths tutor.'},
    {'role': 'user',      'content': 'I keep getting fractions wrong.'},
]
response = ollama.chat(model='gemma3', messages=messages)
print(response.message.content)
```

MENTAL MODEL

Think of the `system` message as the director's note to an actor, the `user` messages as the script the actor responds to, and the `assistant` messages as the lines the actor has already delivered. The model re-reads the whole script every single turn.

3.2 — Writing effective system prompts

The system prompt is the highest-leverage text you'll ever write for a model. A vague one gives vague results; a specific one shapes tone, format, scope and safety all at once. Good system prompts tend to cover four things: **role, behaviour, constraints, and output format.**

Weak system prompt	Strong system prompt
"You are a helpful assistant."	"You are a senior Python reviewer. Point out bugs and style issues concisely, cite the line, and suggest a fix. Never rewrite the whole file."
"Answer questions about cooking."	"You are a friendly chef. Give recipes as a numbered list with metric quantities. If an ingredient is missing, suggest one substitute. Keep replies under 150 words."

A practical template you can adapt:

```
system = """You are {ROLE}.
Your job is to {GOAL}.
Always: {DO RULES}.
Never: {DON'T RULES}.
Format every answer as {FORMAT}."""
```

PRINCIPLES THAT CONSISTENTLY HELP

Be specific over polite · state the desired format explicitly · give boundaries ("never do X") · prefer positive instructions ("do Y") to vague ones · and put the most important rule first. Test changes one at a time so you know what actually moved the needle.

3.3 — Multi-turn conversations: appending to the messages list

Here's the single most important mechanic of conversational AI: **the model has no memory of its own.** Each call to `chat()` is stateless. The illusion of memory comes entirely from *you* sending the whole history back every time. To hold a conversation, you keep one growing `messages` list and, after each exchange, append both the user's message *and* the model's reply.

```
import ollama

messages = [{'role': 'system', 'content': 'You are a helpful travel guide.'}]

def say(user_text):
    # 1. add what the user said
    messages.append({'role': 'user', 'content': user_text})
    # 2. get the model's reply (it sees the FULL history)
    reply = ollama.chat(model='gemma3', messages=messages).message
    # 3. add the reply back so the next turn remembers it
    messages.append({'role': 'assistant', 'content': reply.content})
    return reply.content

print(say('I want to visit Japan in spring.'))
print(say('What should I pack?'))   # model remembers "Japan, spring"
```

The second question never mentions Japan or spring, yet the model answers correctly — because both facts are still in the `messages` list it receives. That three-step loop (append user → call → append assistant) is the beating heart of every chatbot.

> **THE CLASSIC BEGINNER BUG**
>
> Forgetting step 3 — not appending the assistant's reply — gives a model with amnesia that contradicts itself every turn. If your chatbot "forgets" what it just said, this is almost always why.

3.4 — Managing context length (trimming history)

If memory is just a growing list, why not keep everything forever? Two reasons: the model has a finite **context window** (the maximum number of tokens it can read at once), and a longer history means more tokens to process — so slower, heavier responses. By default Ollama uses a context window of **4096 tokens**; you can raise it with the `num_ctx` option, within what the model and your memory support.

```
# Ask for a bigger context window for this call (if the model supports it)
ollama.chat(model='gemma3', messages=messages, options={'num_ctx': 8192})
```

Because you can't grow context forever, real chatbots **trim** old turns. The golden rule: **always keep the system message**, then keep only the most recent N exchanges.

```
def trim(messages, keep_pairs=6):
    """Keep the system prompt + the last `keep_pairs` user/assistant turns."""
    system = [m for m in messages if m['role'] == 'system']
    rest   = [m for m in messages if m['role'] != 'system']
    return system + rest[-keep_pairs * 2:]   # 2 messages per turn
```

SMARTER STRATEGIES

Trimming by count is the simplest approach. More advanced options include *summarising* old turns into a single system note ("Earlier the user said…"), or counting tokens and trimming to a budget. Start with simple trimming — it's enough for most tools.

3.5 — Few-shot prompting

Sometimes describing what you want isn't enough — it's easier to *show* the model. Few-shot prompting means putting a handful of example input/output pairs into the conversation before the real question. The model picks up the pattern and copies it. You do this by seeding the messages list with fake `user` / `assistant` example turns.

```
messages = [
    {'role': 'system', 'content': 'Classify each review as POSITIVE or NEGATIVE.'},
    # --- examples (the "shots") ---
    {'role': 'user',      'content': 'The food was cold and slow.'},
    {'role': 'assistant', 'content': 'NEGATIVE'},
    {'role': 'user',      'content': 'Absolutely loved every bite!'},
    {'role': 'assistant', 'content': 'POSITIVE'},
    # --- the real question ---
    {'role': 'user',      'content': 'The staff were rude but the view was nice.'},
]
print(ollama.chat(model='gemma3', messages=messages).message.content)
NEGATIVE
```

The examples taught the model the exact label vocabulary and output style — no long explanation needed. Few-shot is the fastest way to lock in a consistent format.

> **ZERO-SHOT VS FEW-SHOT**
>
> **Zero-shot** = just ask, no examples. **Few-shot** = ask after showing a few examples. Use few-shot when the task has a specific format, an unusual label set, or a style the model keeps getting slightly wrong.

3.6 — Chain-of-thought (CoT) prompting

For problems involving reasoning — maths, logic, multi-step planning — models are far more accurate when they're allowed to **work through the steps** before giving a final answer, rather than blurting one out. This is chain-of-thought prompting. The classic trigger is simply asking the model to "think step by step".

```
prompt = """A shop sells pens at 3 for £2. I have £10.
How many pens can I buy? Think step by step, then give the final number."""

print(ollama.generate(model='gemma3', prompt=prompt).response)
Step 1: One pen costs £2 / 3 = £0.667.
Step 2: £10 / £0.667 = 15 pens.
Final answer: 15 pens.
```

"THINKING" MODELS DO THIS NATIVELY

Reasoning models such as `deepseek-r1` and `gpt-oss` (introduced in Phase 1) perform chain-of-thought internally and can expose it via Ollama's "thinking" feature. For ordinary chat models, you trigger it yourself with the prompt — and if you only want the final answer in your program, ask the model to put it on a clearly labelled last line you can parse.

3.7 — Structured output: replying in JSON

To use a model's output in real software, you usually need machine-readable data, not prose. Ollama supports **structured outputs** via the `format` parameter, which forces the reply to be valid JSON. There are two levels.

Level 1 — plain JSON mode

Pass `format='json'` and the response is guaranteed to be valid JSON. Tell the model in the prompt what fields you want.

```
r = ollama.chat(
    model='gemma3',
    messages=[{'role': 'user',
               'content': 'Give the capital and population of Japan as JSON.'}],
    format='json',
)
print(r.message.content)
{"capital": "Tokyo", "population": 125000000}
```

Level 2 — enforce an exact schema with Pydantic

For real reliability, define a Pydantic model, pass its JSON schema to `format`, then validate the reply straight back into your typed object. This is the recommended, production-grade approach.

```
from ollama import chat
from pydantic import BaseModel

class Country(BaseModel):
    name: str
    capital: str
    languages: list[str]

response = chat(
    model='gemma3',
    messages=[{'role': 'user', 'content': 'Tell me about Canada.'}],
    format=Country.model_json_schema(),     # enforce the schema
    options={'temperature': 0},            # 0 = deterministic
)

country = Country.model_validate_json(response.message.content)
print(country.capital, country.languages)
Ottawa ['English', 'French']
```

TWO TIPS FOR ROCK-SOLID JSON

(1) Set `temperature` to `0` for the most deterministic, schema-faithful output. (2) It also helps to mention the desired structure in the prompt itself, grounding the model alongside the schema. Validation with `model_validate_json()` then guarantees your program only ever sees well-formed data.

3.8 — Prompt templates: reusable builder functions

Once your prompts get good, you'll want to reuse them without copy-pasting. The Pythonic answer is a **template function**: a function that takes variables and returns a fully-formed messages list or prompt string. This keeps prompt text in one place, makes it testable, and prevents subtle drift.

```
def summarise_prompt(text, style='a single sentence'):
    """Return a messages list that asks for a summary in a given style."""
    return [
        {'role': 'system', 'content': 'You are a precise summariser. No preamble.'},
        {'role': 'user',
         'content': f'Summarise the following in {style}:\n\n{text}'},
    ]

# Reuse it anywhere, with different inputs
article = '... long article text ...'
msgs = summarise_prompt(article, style='three bullet points')
print(ollama.chat(model='gemma3', messages=msgs).message.content)
```

WHY THIS MATTERS AT SCALE

When a prompt lives in a function, you can improve it once and every caller benefits, write tests against it, swap models without touching prompt text, and even load templates from files. This is the bridge from "scripting" to "building real applications".

HANDS-ON PROJECT · PHASE 3

Persona Tutor with Memory — a conversational CLI

This project combines every technique in the phase into one coherent tool: a terminal chat assistant that adopts a **persona via a system prompt**, holds a real **multi-turn conversation** by appending to the messages list, **trims history** automatically to stay within budget, and includes a special `/quiz` command that uses **structured JSON output** to generate a clean quiz question. (Streaming and async come in Phase 4 — this version answers in one piece.)

The complete script — `persona_tutor.py`

```
# persona_tutor.py — a multi-turn tutor with memory + structured quiz
import ollama
from pydantic import BaseModel

MODEL = 'gemma3'

# --- structured output schema for the /quiz command (3.7) ---
class QuizQuestion(BaseModel):
    question: str
    options: list[str]
    answer: str

# --- prompt-template function (3.8) ---
def build_system(subject):
    return {'role': 'system', 'content':
        f'You are a patient {subject} tutor. Explain simply, '
        f'use short examples, and never give more than 120 words.'}

# --- history trimming (3.4) ---
def trim(messages, keep_pairs=6):
    system = [m for m in messages if m['role'] == 'system']
    rest   = [m for m in messages if m['role'] != 'system']
    return system + rest[-keep_pairs * 2:]

def make_quiz(subject):
    """Use structured output to get a clean quiz question."""
    r = ollama.chat(
        model=MODEL,
        messages=[{'role': 'user',
                   'content': f'Write one multiple-choice {subject} question as
JSON.'}],
        format=QuizQuestion.model_json_schema(),
        options={'temperature': 0},
    )
    q = QuizQuestion.model_validate_json(r.message.content)
    print(f'\nQUIZ: {q.question}')
```

```
    for i, opt in enumerate(q.options):
        print(f'  {chr(65 + i)}. {opt}')
    input('Your answer (press Enter to reveal): ')
    print(f'Correct answer: {q.answer}\n')

def main():
    subject = input('Tutor subject (e.g. Python, biology): ') or 'general'
    messages = [build_system(subject)]
    print(f'\n{subject.title()} tutor ready. Type /quiz for a question, /bye to
quit.\n')

    while True:
        user = input('you > ').strip()
        if user == '/bye':
            break
        if user == '/quiz':
            make_quiz(subject)
            continue

        # multi-turn loop (3.3): append user, call, append assistant
        messages.append({'role': 'user', 'content': user})
        reply = ollama.chat(model=MODEL, messages=messages).message.content
        messages.append({'role': 'assistant', 'content': reply})
        messages = trim(messages)          # keep history in budget
        print(f'tutor > {reply}\n')

if __name__ == '__main__':
    main()
```

Run it

```
python persona_tutor.py
```

EXTEND IT YOURSELF

(1) Add a `/persona` command that swaps the system prompt mid-chat. (2) Add a few-shot example pair to `make_quiz` so questions match a house style. (3) Replace count-based trimming with a version that summarises old turns into a single system note.

What you achieved

You used all three roles, wrote a parameterised system prompt via a template function, ran a genuine multi-turn conversation with working memory, kept that memory inside a context budget with trimming, and pulled clean, validated JSON out of the model with a Pydantic schema. That's the full conversational toolkit.

COMING NEXT — PHASE 4: STREAMING & ASYNC

You'll make responses appear token-by-token with `stream=True`, build a streaming terminal chatbot, and use the `AsyncClient` with `asyncio` for concurrent, responsive applications — including async streaming.